

Taller: Comunicación Segura — TLS sobre TCP vs DTLS sobre UDP

Curso: Sistemas Distribuidos

Duración: 120 minutos

Modalidad: Parejas o individual

Prerrequisito: Taller de Sockets TCP y UDP con Wireshark

Objetivos

Al finalizar este taller, el estudiante será capaz de:

- Generar certificados autofirmados con OpenSSL para uso en laboratorio.
- Implementar un servidor y un cliente TLS (TCP seguro) en Python.
- Implementar un servidor y un cliente DTLS (UDP seguro) en Python.
- Capturar el tráfico de ambos protocolos con Wireshark e identificar los mensajes del handshake.
- Comparar el tráfico seguro con el tráfico en texto plano del taller anterior.
- Explicar por qué los datos dejan de ser legibles en la traza y qué información sigue siendo visible.

1. Contexto

En el taller anterior, implementamos servidores y clientes TCP y UDP que enviaban mensajes en texto plano. Al capturar el tráfico con Wireshark, los mensajes eran completamente legibles — cualquier persona con acceso a la red podía leerlos.

En este taller vamos a agregar una capa de seguridad:

- **TLS (Transport Layer Security)** protege conexiones TCP. Es el protocolo detrás de HTTPS.
- **DTLS (Datagram Transport Layer Security)** protege comunicación UDP. Es esencialmente TLS adaptado para datagramas, donde los paquetes pueden perderse o llegar fuera de orden.

Vamos a reutilizar la estructura básica del taller anterior (mismo formato de mensajes, mismo puerto base) para que la comparación en Wireshark sea directa.

2. Preparación del entorno

Herramientas necesarias (además de las del taller anterior):

- OpenSSL instalado (`openssl version` para verificar). Se requiere versión 1.1.1 o superior.
- Python 3.8 o superior con el módulo `ssl` (viene incluido en la biblioteca estándar).

Generar los certificados:

Antes de comenzar con el código, necesitamos un certificado y una clave privada. En un entorno real estos serían emitidos por una Autoridad Certificadora (CA). Para el laboratorio usaremos un certificado autofirmado.

Ejecute el siguiente comando en la terminal:

```
openssl req -x509 -newkey rsa:2048 -keyout clave.pem -out certificado.pem -days 365 -nodes \
```

3. Parte 1 — TLS sobre TCP

3.1 Qué esperar ver en Wireshark

Comparado con el taller anterior, la secuencia de TCP sigue siendo visible (SYN, SYN-ACK, ACK), pero después del handshake TCP aparece un **handshake TLS** adicional antes de que se transmitan datos. En Wireshark verá algo como:

CLIENTE	SERVIDOR
----- SYN ----->	handshake TCP
<----- SYN-ACK -----	(igual que antes)
----- ACK ----->	
----- ClientHello ----->	handshake TLS
<----- ServerHello, Certificate -----	(nuevo)
<----- ServerKeyExchange, Done -----	
----- ClientKeyExchange ----->	
----- ChangeCipherSpec, Finished -->	
<----- ChangeCipherSpec, Finished ---	

4. Parte 2 — DTLS sobre UDP

4.1 Sobre DTLS

DTLS (Datagram TLS) resuelve el problema de encriptar tráfico UDP. No podemos usar TLS directamente sobre UDP porque TLS asume un transporte ordenado y confiable. DTLS adapta TLS para datagramas:

- Añade números de secuencia explícitos a cada registro.
- Incluye su propio mecanismo de retransmisión para el handshake (ya que UDP no retransmite).
- Cada datagrama se encripta de forma independiente — si se pierde uno, los demás siguen siendo descifrables.

4.2 Qué esperar ver en Wireshark

A diferencia de TCP+TLS, aquí **no hay handshake TCP** (no hay SYN/SYN-ACK/ACK).

5. Parte 3 — Comparación

5.1 Actividad: comparar las cuatro capturas

Si conservan las capturas del taller anterior (`captura_tcp.pcap` y `captura_udp.pcap`), pueden abrir las cuatro capturas en Wireshark y compararlas directamente.

Ejercicio guiado: Para cada protocolo (TCP plano, UDP plano, TLS, DTLS), busque en la traza y anote:

Aspecto	TCP plano	UDP plano	TLS (TCP)	DTLS (UDP)
¿Cuántos paquetes antes del primer dato?				
¿El contenido del mensaje es legible?				

6. Cuestionario

Responda las siguientes preguntas basándose en las trazas capturadas.

Sobre TLS:

1. En la captura TLS, ¿cuántos paquetes hay entre el último ACK del handshake TCP y el primer paquete de "Application Data"? Estos paquetes corresponden al handshake TLS. Liste los tipos de mensaje que observa.
2. Abra el paquete ClientHello y expanda los detalles en Wireshark. ¿Qué información envía el cliente en este mensaje? ¿Está encriptada esta información?
3. Abra el paquete ServerHello. ¿Qué versión de TLS y qué cipher suite se negociaron? Anótelos.
4. Después del handshake, seleccione un paquete de "Application Data". ¿Puede

7. Entregables

- `certificado.pem` y `clave.pem` generados.
- `tls_servidor.py` y `tls_cliente.py` funcionando.
- `dtls_servidor.py` y `dtls_cliente.py` funcionando (o evidencia de ejecución con `openssl s_server / s_client`).
- `captura_tls.pcap` con la sesión TLS completa.
- `captura_dtls.pcap` con la sesión DTLS completa.
- Este documento con el cuestionario respondido.
- Capturas de pantalla de los paquetes ClientHello (TLS y DTLS) expandidos en Wireshark.

8. Rúbrica

Componente	Peso
Código TLS funcional (servidor + cliente)	20%
Código DTLS funcional (servidor + cliente)	20%
Capturas .pcap con tráfico completo	20%
Cuestionario respondido	30%
Tabla comparativa completa (pregunta 10)	10%

9. Recursos adicionales

- RFC 8446 — TLS 1.3: <https://www.rfc-editor.org/rfc/rfc8446>
- RFC 9147 — DTLS 1.3: <https://www.rfc-editor.org/rfc/rfc9147>
- Documentación del módulo `ssl` de Python:
<https://docs.python.org/3/library/ssl.html>
- Documentación de `openssl s_server` :
https://www.openssl.org/docs/man3.0/man1/openssl-s_server.html
- Documentación de `openssl s_client` :
https://www.openssl.org/docs/man3.0/man1/openssl-s_client.html
- Wiki de Wireshark sobre TLS: <https://wiki.wireshark.org/TLS>
- Wiki de Wireshark sobre DTLS: <https://wiki.wireshark.org/DTLS>